

Microservice-based in-network security framework for FPGA NICs

Lasani Hussain*, Mayank Rawat*, Neeraj Kumar Yadav, Sumit Darak, Praveen Tammana†, Rinku Shah
{IIIT Delhi, IIT Hyderabad†}, India

{lasani21042, mayank18049, neerajy, sumit}@iiitd.ac.in, praveent@cse.iith.ac.in, rinku@iiitd.ac.in

I. INTRODUCTION AND MOTIVATION

Recent advancements in programmable network hardware such as smartNICs (e.g., NPU, DPU, FPGA) ¹ and programmable switches (e.g., Intel Tofino) have created opportunities to save precious CPU cycles and achieve low latencies by offloading applications to these devices [1]–[3].

Some example applications offloaded to smartNIC include: (a) Cloud infrastructure services such as failover manager [4], load balancer [5], consensus manager [6], and replication manager [6], (b) Cloud applications such as key-value stores [7] and serverless application infrastructure [4], [5], and (c) 4G/5G mobile packet core functions [8]–[11].

Offloading the applications to smartNIC implies that we must also offload the security algorithms traditionally supported by the host. Otherwise, we need to send the application messages to the host's VM/container for crypto processing operations, negating offload benefits.

SmartNICs such as Cavium's Xpliant [12]), Netronome's Agilio CX [13], and Nvidia's Bluefield [14]) come with fixed-function accelerators for popular security algorithms such as AES. Several solutions propose reconfigurable security algorithm implementation for programmable network hardware. For example, HalfSipHash [15], [16] is implemented on Intel Tofino; ChaCha [17] is implemented on Netronome smartNIC; and Advanced Encryption Standard (AES) [18]–[20] is implemented on Intel Tofino and FPGA hardware.

The existing fixed-function and reconfigurable security acceleration solutions focus on individual security algorithm implementation. In practice, a server application or a mobile network must support multiple security algorithms for confidentiality, integrity, and authentication. For example, the TLS cipher suite [21], [22] supports several public-key algorithms (e.g., RSA, ECDSA), secret-key algorithms (e.g., ChaCha20, AES-GCM, AES-CBC), and authentication and hash algorithms (e.g., Poly1305, SHA). Similarly, the 5G standards [23], [24] support security algorithms such as NEA0, 128-NEA1 (SNOW 3G), 128-NEA2 (128-bit AES in CTR mode), 128-NEA3 (ZUC-based), and integrity, viz, NIA0, 128-NIA1 (SNOW 3G), 128-NIA2 (AES in CMAC mode), 128-NIA3 (ZUC-based). Therefore, the existing solutions that focus on a single security algorithm miss out on acceleration

opportunities across security algorithms. The state-of-the-art does not support performance isolation among multiple cloud tenants (or 5G network slices) and auto-scaling.

II. PROBLEM STATEMENT

Microservice architectures [25] provide benefits such as independent scaling of bottleneck functions, code reusability, and ease of management. Inspired by these benefits, we propose to migrate from monolithic architecture to microservice security architecture. In this work, we propose to design and implement an in-network, microservice-based security framework for offloaded applications by leveraging FPGA NICs. Our goal is to decompose the TLS cipher suite and 4G/5G security algorithms into independent crypto functions (i.e., microservices). We aim to design a programmable system that supports near line-rate processing for signaling and data messages. It should judiciously utilize the FPGA hardware to maximally pack crypto algorithms and support the auto-scaling of crypto microservices based on the incoming traffic characteristics.

We plan to identify the common crypto functions across security algorithms, design microservice chains for request (e.g., encrypt, decrypt, authenticate) processing. Our final objective is to provide microservice-based crypto function APIs to the cloud/telecom application programmers to help them accelerate the security of offloaded applications.

Some of the use cases that would benefit from our offload framework include: (a) security procedure offload for server applications that use HTTPS/TLS, 4G/5G mobile edge and core network security procedures, (b) network functions that terminate/initiate secure connections (e.g., IPsec gateway, L4 load balancer, stateful firewall, intrusion detection system), and (c) secure communication between microservices in a serverless function chain. Our work is generalizable across security use cases, but we plan to demonstrate our prototype for the HTTPS/TLS and 4G/5G security use cases.

Challenges. We have identified the following challenges in designing our proposed in-network security framework. (1) Packing multiple crypto algorithms on a single FPGA hardware to efficiently utilize FPGA hardware resources, (2) Designing an in-network system that can perform crypto processing at line rates (up to 100Gbps) for messages as small as 64 bytes. Typical control/signaling messages are small, (3) Ensuring performance isolation, fairness, and QoS guarantees for workloads with different characteristics and requirements.

* Student authors with equal contribution.

¹Abbreviations — NPU: Network processing unit, DPU: Data processing unit, FPGA: Field Programmable Gate Arrays

III. PROPOSED APPROACH

We propose the following approach to accomplish our research objectives.

Identification of independent functions for security algorithms. The first step is to decompose the security algorithms into independent microservices to improve code reusability and leverage opportunities for independent scaling.

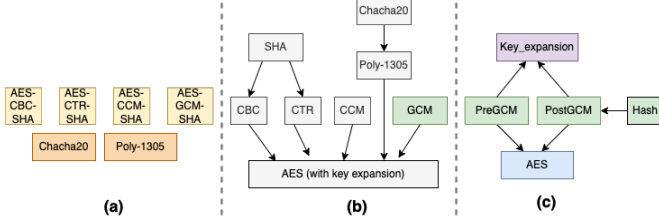


Figure 1: (a) TLS1.2/TLS1.3 cipher suite (data path), (b) Microservice design for TLS data path security algorithms, (c) Microservice design for AES-GCM.

Figure 1 (a) shows the TLS cipher suite algorithms. Figure 1 (b) shows the decomposition of the TLS cipher suite algorithms into microservices with the help of a dependency graph (DAG). The notation $A \rightarrow B$ indicates that microservice A depends on B . For example, SHA depends on CBC, and CBC depends on AES.

We now discuss the analysis that led to microservice decomposition. Figure 1(b) shows that AES and SHA are decomposed as microservices as they are common across multiple security algorithms. The key expansion function of AES is invoked only during the first packet of the flow, after which the function remains unused. To save the hardware resource during function scaling, we split the key expansion function as a microservice and add corresponding dependencies. (see figure 1(c)) GCM is further decomposed into three microservices to reuse AES across security algorithms (see figure 1(c)).

The proposed design helps hardware resource reuse, eases maintenance and debugging of the code, and helps fine-grained control for management components such as scaling. The construction of microservice design for 5G security algorithms is work-in-progress.

Hardware design to support line-rate processing. We propose designing the dynamic chaining algorithm for security algorithms that leverages the knowledge of common microservices for efficient hardware utilization. We plan to design a scheduler incorporating dynamic chaining and load balancing to provide near-line-rate performance.

Controller design for dynamic scaling. We plan to design a controller that monitors the incoming traffic characteristics, collects the hardware resource utilization statistics, learns near-future computing needs, and dynamically reconfigures the FPGA hardware without impacting other processing units.

IV. INITIAL RESULTS

Experimental setup. We have implemented the following designs for evaluation. (1) *Container (without AES-NI)*, (2)

Container (AES-NI). We implemented AES-GCM using the TLS library within a docker container. The server CPU uses AES-NI accelerators to speed up AES operations, which acts as our host-accelerated design. We disabled AES-NI for our baseline design, (3) *PS (FPGA_CPU)*. We implemented AES-GCM for the Processing System (PS), i.e., the CPU, on the FPGA board to avoid traveling the host network stack, (4) *PS+PL (FPGA_CPU+FPGA)*. We implemented the monolithic AES-GCM algorithm from scratch using the microservice design principles on PL (Programmable Logic) of the FPGA, Zynq UltraScale+ MPSoc ZCU106 [26]. Our monolithic AES-GCM spanned around 1K lines of hardware code that includes the high-level synthesis (HLS) source for AES-GCM, HLS testbench for verification and the driver code for PS-PL communication. (5) *PS+PL_scaled (Xilinx SN1000 [27])*. We extrapolated the performance for the FPGA NIC Xilinx SN1000 to estimate the gains for production-grade FPGAs.

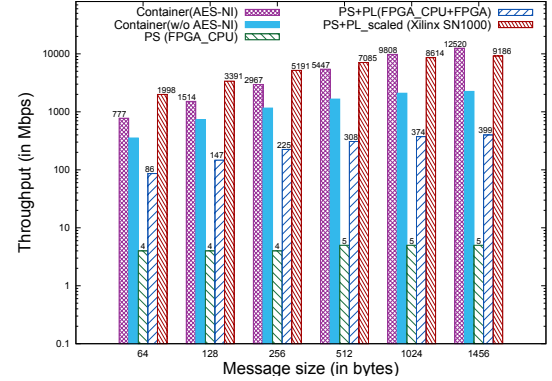


Figure 2: Message size vs. Throughput.

Message size (in bytes)	Latency (in μ s)		
	Container (AES-NI) Compute	Container (AES-NI) End-to-end	FPGA End-to-end
64	0.65	192.57	5.89
128	0.67	194.65	6.94
256	0.69	219.18	9.07
512	0.75	231.17	13.29
1024	0.83	243.21	21.87
1464	0.93	253.73	29.16

Table I: Latency: FPGA and host accelerator designs.

Evaluation. Figure 2 and Table I show AES-GCM performance in terms of throughput and latency, respectively. The latency metrics are computation time and end-to-end latency. The monolithic AES-GCM FPGA-based solution reduces end-to-end latency from 88.5% to 97.3% compared to host-based AES-NI accelerators. The computation time with AES-NI contributes to less than 1% of end-to-end latency; the rest is spent on network stack processing. This makes host-based accelerators a bad choice for in-network workloads.

The accelerated monolithic design, PS+PL_scaled, performs better than the AES-NI CPU accelerators for packet size ≤ 512 bytes. The performance suffers for large packet sizes because the scheduler runs on the low-frequency CPU (PS). We plan to build a PL-based scheduler to improve the throughput. We hypothesize that our system will scale better with the microservice design.

REFERENCES

- [1] D. Moro, M. Peuster, H. Karl, and A. Capone, "Fop4: Function offloading prototyping in heterogeneous and programmable network scenarios," in *2019 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2019, pp. 1–6.
- [2] D. Firestone, A. Putnam, S. Mundkur, D. Chiou, A. Dabagh, M. Andrewartha, H. Angepat, V. Bhanu, A. Caulfield, E. Chung, H. K. Chandrappa, S. Chaturmohta, M. Humphrey, J. Lavier, N. Lam, F. Liu, K. Ovtcharov, J. Padhye, G. Popuri, S. Raindel, T. Sapre, M. Shaw, G. Silva, M. Sivakumar, N. Srivastava, A. Verma, Q. Zuhair, D. Bansal, D. Burger, K. Vaid, D. A. Maltz, and A. Greenberg, "Azure accelerated networking: Smartnics in the public cloud," in *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'18. USA: USENIX Association, 2018, p. 51–64.
- [3] D. Kim, S. Lee, and K. Park, "A case for smartnic-accelerated private communication," in *4th Asia-Pacific Workshop on Networking*, ser. APNet '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 30–35. [Online]. Available: <https://doi.org/10.1145/3411029.3411034>
- [4] M. Liu, S. Peter, A. Krishnamurthy, and P. M. Phothilimthana, "E3: Energy-Efficient microservices on SmartNIC-Accelerated servers," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. Renton, WA: USENIX Association, Jul. 2019, pp. 363–378. [Online]. Available: <https://www.usenix.org/conference/atc19/presentation/liu-ming>
- [5] N. Daw, U. Bellur, and P. Kulkarni, "Speedo: Fast dispatch and orchestration of serverless workflows," in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 585–599. [Online]. Available: <https://doi.org/10.1145/3472883.3486982>
- [6] M. Liu, T. Cui, H. Schuh, A. Krishnamurthy, S. Peter, and K. Gupta, "Offloading distributed applications onto smartnics using ipipe," in *Proceedings of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 318–333. [Online]. Available: <https://doi.org/10.1145/3341302.3342079>
- [7] X. Jin, X. Li, H. Zhang, R. Soulé, J. Lee, N. Foster, C. Kim, and I. Stoica, "Netcache: Balancing key-value stores with fast in-network caching," in *Proceedings of the SOSP*, 2017.
- [8] R. Shah, V. Kumar, M. Vutukuru, and P. Kulkarni, "Turboepc: Leveraging dataplane programmability to accelerate the mobile packet core," in *Proceedings of the Symposium on SDN Research*, ser. SOSR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 83–95. [Online]. Available: <https://doi.org/10.1145/3373360.3380839>
- [9] MacDavid, Robert and Cascone, Carmelo and Lin, Pingping and Padmanabhan, Badhrinath and Thakur, Ajay and Peterson, Larry and Rexford, Jennifer and Sunay, Oguz, "A P4-Based 5G User Plane Function," in *Proceedings of the ACM SIGCOMM Symposium on SDN Research (SOSR)*, 2021.
- [10] A. Bose, D. Maji, P. Agarwal, N. Unhale, R. Shah, and M. Vutukuru, "Leveraging programmable dataplanes for a high performance 5g user plane function," in *5th Asia-Pacific Workshop on Networking (APNet 2021)*, ser. APNet 2021. New York, NY, USA: Association for Computing Machinery, 2022, p. 57–64.
- [11] A. Bose, S. Kirtikar, S. Chirumamilla, R. Shah, and M. Vutukuru, "Accelupf: Accelerating the 5g user plane using programmable hardware," in *Proceedings of the Symposium on SDN Research*, ser. SOSR '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1–15. [Online]. Available: <https://doi.org/10.1145/3563647.3563651>
- [12] Cavium, "Cavium LiquidIO® Server Adapter Family," <https://datasheet.octopart.com/CN6130-110SV-G-Cavium-Networks-datasheet-26366670.pdf>, 2013, [Online; accessed 16-March-2022].
- [13] N. Systems, "vEPC Acceleration Using Agilio SmartNICs," https://www.netronome.com/media/documents/SB_vEPC.pdf, 2017.
- [14] "Nvidia bluefield-3 dpu programmable data center infrastructure on-a-chip," <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf>, 2021.
- [15] S. Yoo and X. Chen, "Secure keyed hashing on programmable switches," in *Proceedings of the ACM SIGCOMM 2021 Workshop on Secure Programmable Network Infrastructure*, ser. SPIN '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 16–22. [Online]. Available: <https://doi.org/10.1145/3472873.3472881>
- [16] D. Scholz, A. Oeldemann, F. Geyer, S. Gallenmüller, H. Stubbe, T. Wild, A. Herkersdorf, and G. Carle, "Cryptographic hashing in p4 data planes," in *2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019, pp. 1–6.
- [17] S. Zuveria Kottur, K. Kadiyala, P. Tammana, and R. Shah, "Implementing chacha based crypto primitives on programmable smartnics," in *Workshop on Formal Foundations and Security of Programmable network Infrastructure (FFSPIN'22)*, ser. FFSPIN'22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3528082.3544833>
- [18] X. Chen, "Implementing aes encryption on programmable switches via scrambled lookup tables," in *Proceedings of the Workshop on Secure Programmable Network Infrastructure*, ser. SPIN '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 8–14. [Online]. Available: <https://doi.org/10.1145/3405669.3405819>
- [19] P. Visconti, R. Velazquez, S. Capoccia, and R. de Fazio, "High-performance aes-128 algorithm implementation by fpga-based soc for 5g communications," *International Journal of Electrical & Computer Engineering (2088-8708)*, vol. 11, no. 5, 2021.
- [20] P. Visconti, R. Velazquez, C. D.-V. Soto, and R. de Fazio, "Fpga based technical solutions for high throughput data processing and encryption for 5g communication: A review," *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, vol. 19, no. 4, pp. 1291–1306, 2021.
- [21] T. Dierks and E. Rescorla. (2008) The Transport Layer Security (TLS) Protocol, Version 1.2. <https://www.rfc-editor.org/rfc/rfc5246>.
- [22] E. Rescorla. (2018) The Transport Layer Security (TLS) Protocol, Version 1.3. <https://www.rfc-editor.org/rfc/rfc8446>.
- [23] (2019) Etsi ts 133 401 v15.7.0. security architecture and procedures for 5g system). [Online]. Available: https://www.etsi.org/deliver/etsi_ts/133400_133499/133401/15.07.00_60/ts_133401v150700p.pdf
- [24] (2019) Etsi ts 133 501 v15.4.0. security architecture and procedures for 5g system). [Online]. Available: https://www.etsi.org/deliver/etsi_ts/133500_133599/133501/15.04.00_60/ts_133501v150400p.pdf
- [25] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022.
- [26] (2021) Zynq UltraScale+ MPSoC ZCU106 Evaluation Kit. <https://www.xilinx.com/products/boards-and-kits/zcu106.html>.
- [27] (2023) Alveo SN1000 SmartNIC Accelerator Card. <https://www.xilinx.com/products/boards-and-kits/alveo/sn1000.html>.